

Experiment No.	2
Experiment Title	Email Spam/Ham Classification using Naïve Bayes and KNN 
Student Name	Poojashree V
Register Number	3122247001043
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	14th July 2026

1 Objective

This experiment builds spam classifiers for the Spambase e-mail dataset using Naïve Bayes and K-Nearest Neighbours (KNN), and compares how the two families perform against each other. We evaluate the Gaussian, Multinomial, and Bernoulli variants of Naïve Bayes on the same train/test split so their relative strengths can be judged fairly. For KNN, we look at how varying the number of neighbours (k) changes classification accuracy, and try to pin down a reasonable value of k for this data. We also compare KDTree and BallTree, the two data structures commonly used to speed up neighbour search in KNN, and check whether GridSearchCV or RandomizedSearchCV does a better job tuning KNN's hyperparameters. Finally, we look at both the theoretical and the actually-measured training/prediction times for each algorithm, and validate everything with 5-fold cross-validation rather than trusting a single split.

2 Problem Statement

Each e-mail in this dataset is represented as a 57-dimensional numeric feature vector: 48 relative frequencies of specific words (`word_freq_*`), 6 relative frequencies of specific characters (`char_freq_*`), and 3 statistics describing runs of consecutive capital letters (`capital_run_length_*`). The task is to predict the binary label `class` $\in \{0, 1\}$, where 1 means *spam* and 0 means *ham* (a legitimate e-mail). This is a standard supervised, binary classification problem, and we build two quite different families of classifiers to attack it: a generative probabilistic model (Naïve Bayes, in three flavours suited to different assumptions about the features) and a non-parametric, instance-based model (KNN). The point of this report isn't really to chase the single highest accuracy number – it's to understand *why* each algorithm behaves the way it does on this particular feature space, and how choices like distance metric, neighbour-search structure, and hyperparameter search strategy trade accuracy off against computational cost.

3 Dataset Description

Dataset Name	Spambase Dataset
Dataset Source	Kaggle – https://www.kaggle.com/datasets/somesh24/spambase (mirror of the UCI Machine Learning Repository Spambase dataset)
Number of Samples	4601 e-mail instances
Number of Features	57 numeric features (48 <code>word_freq_*</code> , 6 <code>char_freq_*</code> , 3 <code>capital_run_length_*</code>)

Number of Classes	2 – <code>class = 1</code> (spam), <code>class = 0</code> (ham). This matches the classic Spambase composition of 1,813 spam ($\approx 39.4\%$) and 2,788 ham ($\approx 60.6\%$) instances, which lines up with the 4601-row count we loaded and is visible in the class-distribution plot in Section 4.
Missing Values	None found – <code>df.info()</code> reported 4601 non-null entries across all 58 columns. Even so, the pipeline still coerces every feature column to numeric with <code>pd.to_numeric(..., errors="coerce")</code> and fills any resulting NaN with the column median, mainly as a safeguard. In practice this step turned out to be a no-op on the real data.
Train-Test Split	80% / 20% stratified split (<code>train_test_split, random_state=42, stratify=y</code>) $\rightarrow$ 3680 training samples, 921 test samples, keeping the 39.4%/60.6% class ratio consistent in both.

4 Exploratory Data Analysis

4.1 Class Distribution

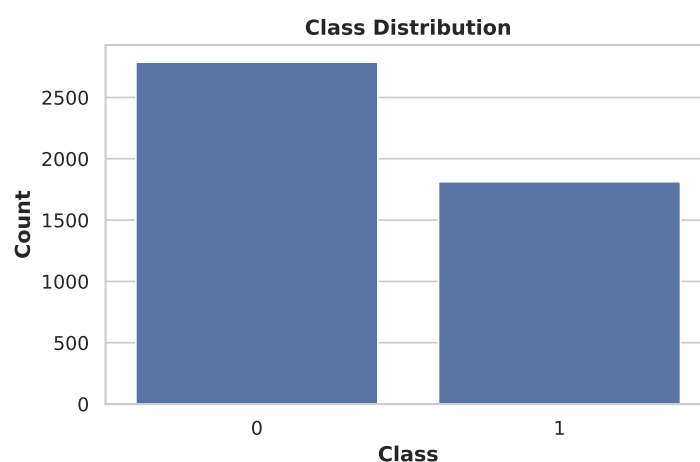


Figure 1: Class distribution of the Spambase dataset (0 = ham, 1 = spam).

Inference: From Figure 1 we can see the dataset is moderately imbalanced, with ham outnumbering spam by roughly 3:2. That’s not severe enough to make raw accuracy meaningless, but it’s also not something to ignore – a classifier that just predicted “ham” every time would already land close to 60% accuracy without learning anything. That’s exactly why we track Precision, Recall, F1 and ROC-AUC alongside accuracy for every model in this report. It ends up mattering most for Bernoulli NB (Section 8.1), whose raw accuracy of 0.6482 is barely above that trivial baseline once you look closely.

4.2 Correlation Heatmap

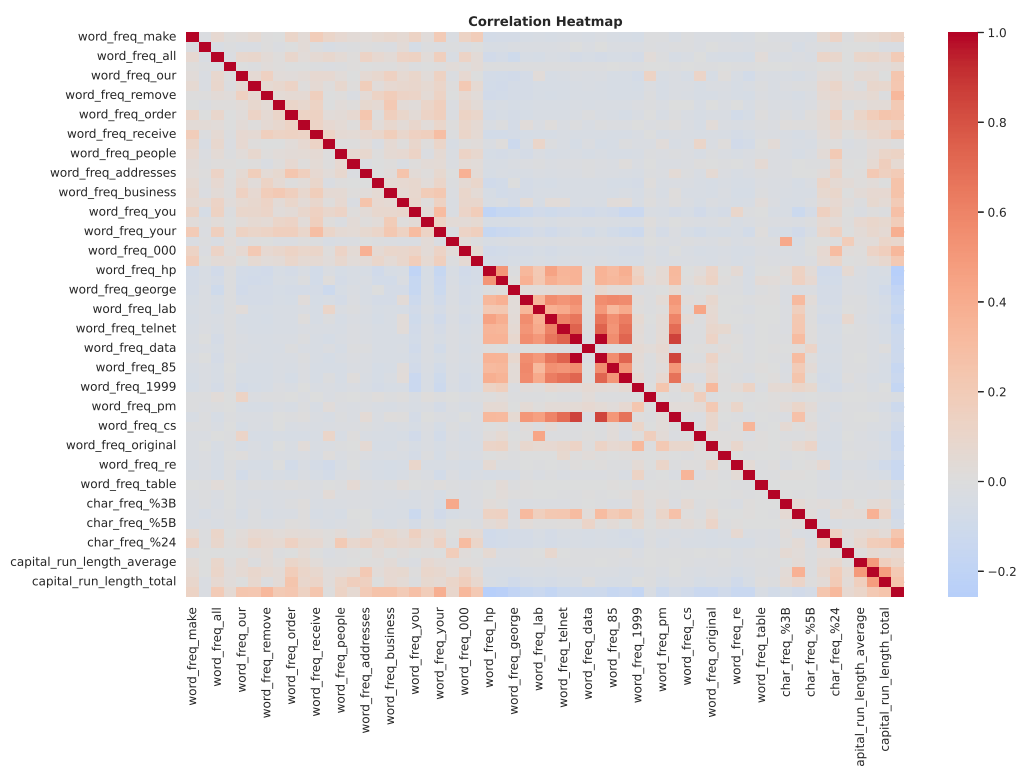


Figure 2: Pearson correlation heatmap across all 58 columns of the preprocessed dataset.

Inference: Looking at Figure 2, the 48 `word_freq_*` columns and the 6 `char_freq_*` columns each seem to form a loosely correlated block. That’s not too surprising – promotional/spam vocabulary (words like “free”, “money”, “credit”) tends to show up together in the same messages. The three `capital_run_length_*` features are much more tightly correlated with each other, which makes sense since all three are really just different ways of describing bursts of capitalisation, so they carry a fair bit of overlapping rather than independent information. We also notice that no single feature correlates strongly with the class label on its own – correlations are weak-to-moderate across the board. That’s actually favourable for Naïve Bayes’ conditional-independence assumption, but it also explains why no individual feature is a reliable spam signal by itself; the model has to combine a lot of weak signals to get anywhere.

4.3 Feature Distributions (Top-4 Correlated Features)

The four features most strongly correlated (by absolute value) with the target label turned out to be `word_freq_your`, `word_freq_000`, `word_freq_remove` and `char_freq_%24` (the frequency of the ‘\$’ character; it’s stored as %24, its percent-encoded form, in the source CSV column header).

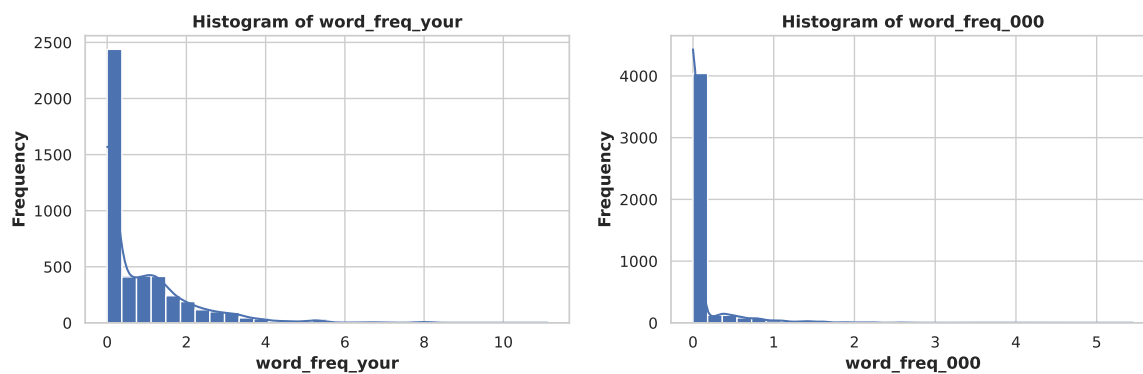


Figure 3: Histograms of word_freq_your and word_freq_000.

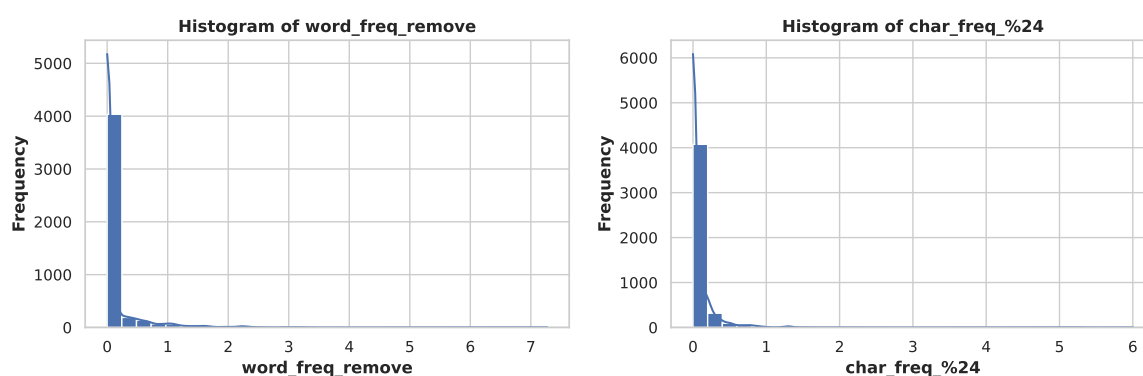


Figure 4: Histograms of word_freq_remove and char_freq_%24.

Inference: All four distributions here are heavily right-skewed and zero-inflated – most e-mails contain each of these tokens/characters 0% of the time, with only a small tail of messages using them repeatedly. This is a rough match for what GaussianNB assumes (a per-class normal distribution), and it's probably why Gaussian NB over-predicts spam – high recall (0.9532) but weak precision (0.7178), which we see later in Section 8.1. It's arguably worse for BernoulliNB. Since almost every value sits far below the fixed 0.5 binarisation threshold, even in messages that are genuinely spammy, the binarised features collapse to 0 for most instances. One likely explanation for Bernoulli NB's very low recall (0.1295) is exactly this – the threshold wipes out the signal that made these features useful in the first place.

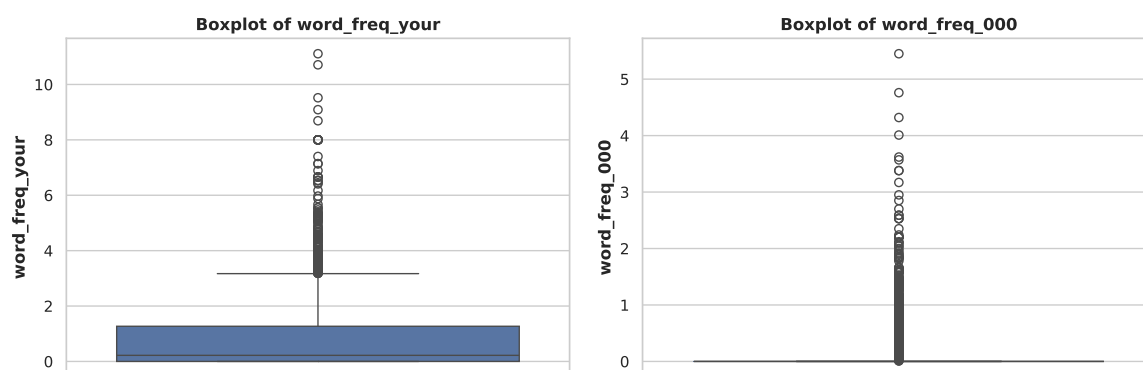


Figure 5: Boxplots of word_freq_your and word_freq_000.

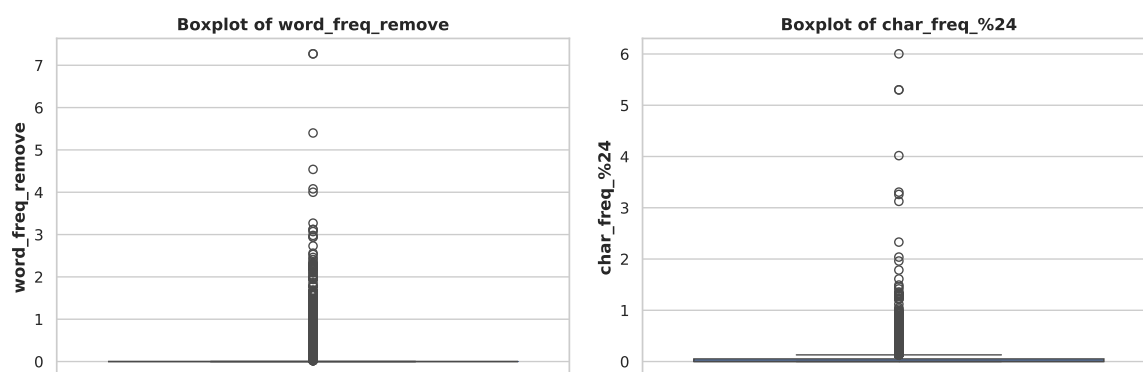


Figure 6: Boxplots of `word_freq_remove` and `char_freq_%24`.

Inference: These boxplots back up what we saw in the histograms – the interquartile box for each of these four features sits squeezed near zero, with a long tail of outliers stretching well past the upper whisker. We were tempted to treat those outliers as noise, but they’re not; they’re basically the spam messages that repeatedly use “your”, numeric strings, the word “remove” (common in fake unsubscribe text), and the “\$” symbol (common in monetary spam). So we left them in and didn’t trim anything before modelling.

5 Data Preprocessing

- **Missing value handling:** We coerced feature columns to numeric (`errors="coerce"`) and filled any resulting NaN with the column median. The target column would have been label-encoded if it were non-numeric, but that wasn’t needed here since `class` was already integer-typed. As noted in Section 3, the raw data had no missing values anyway, so this step was really just a safety net.
- **Encoding:** Not needed – all 57 features and the target are already numeric.
- **Feature scaling:** We applied `MinMaxScaler` (scaling every feature to $[0, 1]$) inside the pipeline for every model except Gaussian NB, which works directly on the raw feature scale since it just estimates a per-class mean and variance rather than depending on distance or non-negative counts. Scaling matters a lot for Multinomial NB (which needs non-negative inputs), for Bernoulli NB (whose `Binarizer(threshold=0.5)` step needs a bounded $[0, 1]$ range), and for all KNN variants, where distance would otherwise be dominated by the large-magnitude `capital_run_length_*` features.
- **Train-test split:** 80/20 stratified split, `random_state=42`, as described in Section 3.
- **Feature engineering:** None beyond scaling/binarising – we used the raw 57 Spambase features as they came.

6 Code Implementation

The core building blocks of the pipeline are shown below (the full code is in the accompanying notebook).

Preprocessing

```

1 target_col = df.columns[-1]
2 if df[target_col].dtype == "object":
3     le = LabelEncoder()
4     df[target_col] = le.fit_transform(df[target_col])
5
6 for col in df.columns[:-1]:
7     df[col] = pd.to_numeric(df[col], errors="coerce")
8 df = df.fillna(df.median(numeric_only=True))

```

Unified Evaluation Function

```

1 def evaluate_pipeline(model, name, X_train, X_test, y_train,
2   y_test, make_plots=True):
3     start_train = time.perf_counter()
4     model.fit(X_train, y_train)
5     train_time = time.perf_counter() - start_train
6
7     start_pred = time.perf_counter()
8     y_pred = model.predict(X_test)
9     pred_time = time.perf_counter() - start_pred
10
11     y_score = model.predict_proba(X_test)[:, 1] if hasattr(model,
12   "predict_proba") \
13       else model.decision_function(X_test)
14
15     acc = accuracy_score(y_test, y_pred)
16     prec = precision_score(y_test, y_pred, zero_division=0)
17     rec = recall_score(y_test, y_pred, zero_division=0)
18     f1 = f1_score(y_test, y_pred, zero_division=0)
19     auc = roc_auc_score(y_test, y_score)
20     return {"Model": name, "Accuracy": acc, "Precision": prec, "
21   Recall": rec,
22   "F1-Score": f1, "ROC-AUC": auc, "Train Time (s)":
23   train_time,
24   "Pred Time (s)": pred_time}

```

Model Pipelines

```

1 gaussian_model = Pipeline([("clf", GaussianNB())])
2 multinomial_model = Pipeline([("scaler", MinMaxScaler()), ("clf",
3   MultinomialNB())])
4 bernoulli_model = Pipeline([
5   ("scaler", MinMaxScaler()),
6   ("bin", Binarizer(threshold=0.5)),
7   ("clf", BernoulliNB())
8 ])

```

Hyperparameter Search Space (KNN)

```

1 param_grid = {
2     "clf__n_neighbors": [1, 3, 5, 7, 9, 11],
3     "clf__weights": ["uniform", "distance"],
4     "clf__algorithm": ["brute", "kd_tree", "ball_tree"],
5     "clf__metric": ["minkowski"],
6     "clf__p": [1, 2]
7 }
```

7 Machine Learning Algorithm

7.1 Naïve Bayes

Naïve Bayes is a generative probabilistic classifier built on Bayes' theorem, with the simplifying (“naïve”) assumption that features are conditionally independent given the class:

$$P(C | X) = \frac{P(X | C)P(C)}{P(X)}, \quad P(X | C) = \prod_{i=1}^d P(x_i | C)$$

The predicted class is $\hat{C} = \arg \max_C P(C) \prod_{i=1}^d P(x_i | C)$. The three variants we use here only differ in what form $P(x_i | C)$ is assumed to take:

- **Gaussian NB:** assumes $x_i | C \sim \mathcal{N}(\mu_C, \sigma_C^2)$, i.e. $P(x_i | C) = \frac{1}{\sqrt{2\pi\sigma_C^2}} \exp\left(-\frac{(x_i - \mu_C)^2}{2\sigma_C^2}\right)$. Works best for continuous, roughly bell-shaped features.
- **Multinomial NB:** treats features as if they're counts/frequencies drawn from a multinomial distribution, $P(x | C) \propto \prod_i p_{C,i}^{x_i}$ (with Laplace smoothing). This tends to suit word-frequency-style features such as `word_freq_*` quite well.
- **Bernoulli NB:** assumes each feature is a binary indicator, $P(x | C) = \prod_i p_i^{x_i} (1 - p_i)^{1-x_i}$, and unlike Multinomial NB, it explicitly penalises the *absence* of a feature too.

Advantages: very fast to train ($O(nd)$) and predict ($O(d)$), works reasonably well even with limited data, and handles high-dimensional feature spaces without much fuss. **Limitations:** the independence assumption almost never holds exactly (Section 4.2 shows correlated feature blocks in this very dataset), and each variant's distributional assumption can be badly mismatched to the actual data – which is basically what happens with Gaussian and Bernoulli NB here.

7.2 K-Nearest Neighbours (KNN)

KNN is a non-parametric, instance-based (“lazy”) learner. It just stores the entire training set, and at prediction time assigns a query point the majority class among its k nearest training points, under some distance metric $d(\cdot, \cdot)$ – typically the Minkowski distance

$$d(x, x') = \left(\sum_{i=1}^d |x_i - x'_i|^p \right)^{1/p}$$

which reduces to Manhattan distance at $p = 1$ and Euclidean distance at $p = 2$. In *weighted* KNN, each neighbour's vote gets weighted by $1/d(x, x')$, so closer neighbours count for more.

Naive (brute-force) prediction computes the distance to every one of the n training points, so it costs $O(nd)$ per query. **KDTree** and **BallTree** are space-partitioning structures built once at training time ($O(n \log n)$) that let neighbour queries be answered in $O(\log n)$ on average, by pruning away large parts of the search space. KDTree recursively splits the space along axis-aligned hyperplanes, which works well in low-to-moderate dimensions, but its pruning power drops off toward brute-force as dimensionality grows – relevant here, since $d = 57$. BallTree instead partitions the data into nested hyperspheres, each defined by a centroid and radius, and tends to hold up better in higher-dimensional or unevenly distributed spaces.

Advantages: no real training phase beyond storing the data, naturally forms non-linear/multi-modal decision boundaries, and only really has two hyperparameters to worry about (k and the distance metric). **Limitations:** prediction gets expensive, especially with brute force, performance is sensitive to feature scaling and to the curse of dimensionality, and the whole training set has to stay in memory.

8 Experimental Setup

Python Version	3.12
Scikit-Learn Version	1.6.1
IDE	Jupyter Notebook
Operating System	Linux (Ubuntu-based)
Hardware	CPU

9 Hyperparameter Selection

We went with the final KNN configuration from GridSearchCV (5-fold CV), since its cross-validated accuracy (0.9163) came out ahead of RandomizedSearchCV's (0.9054):

Parameter	Value
<code>n_neighbors</code>	7
<code>weights</code>	distance
<code>algorithm</code>	brute
<code>metric</code>	minkowski
<code>p</code>	1 (Manhattan distance)

10 Result

10.1 Naïve Bayes Comparison

Metric	Gaussian	Multinomial	Bernoulli
Accuracy	0.8339	0.8958	0.6482
Precision	0.7178	0.9349	0.8545
Recall	0.9532	0.7906	0.1295
F1	0.8189	0.8567	0.2249
ROC-AUC	0.9449	0.9612	0.6135

10.2 KNN Performance vs. k

k	Accuracy	Precision	Recall	F1	ROC-AUC
1	0.8925	0.8607	0.8678	0.8642	0.8882
3	0.9001	0.8817	0.8623	0.8719	0.9343
5	0.9034	0.8848	0.8678	0.8762	0.9444
7	0.8990	0.8792	0.8623	0.8707	0.9476
9	0.8947	0.8757	0.8540	0.8647	0.9513
11	0.8969	0.8829	0.8512	0.8668	0.9537

10.3 GridSearchCV vs. RandomizedSearchCV

Parameter	GridSearchCV	RandomizedSearchCV
Best k	7	3
Metric	minkowski	minkowski
Weights	distance	uniform
Algorithm	brute	kd_tree
CV Accuracy	0.9163	0.9054
Execution Time (s)	18.0774	2.1568

10.4 KDTree vs. BallTree

Both trees were evaluated using the GridSearchCV-selected hyperparameters ($k = 7$, weights=distance, $p = 1$):

Metric	KDTree	BallTree
Accuracy	0.9175	0.9175
Training Time (s)	0.0223	0.0137
Prediction Time (s)	0.2151	0.1795

10.5 5-Fold Cross Validation

Fold	Gaussian NB	Multinomial NB	Bernoulli NB	Best KNN
1	0.8502	0.8665	0.6222	0.8817
2	0.8707	0.8870	0.6315	0.8978
3	0.8554	0.8739	0.6348	0.9293
4	0.8522	0.9076	0.6402	0.9087
5	0.7000	0.8533	0.6207	0.8261
Average	0.8257	0.8776	0.6299	0.8887

10.6 Theoretical Time Complexity

Algorithm	Training	Prediction
Naïve Bayes	$O(nd)$	$O(d)$
KNN (Brute)	$O(1)$	$O(nd)$
KDTree	$O(n \log n)$	$O(\log n)$ avg
BallTree	$O(n \log n)$	$O(\log n)$ avg

10.7 Experimental Time Analysis

Algorithm	Training(s)	Prediction(s)
Gaussian NB	0.006289	0.002403
Multinomial NB	0.005920	0.002284
Bernoulli NB	0.007567	0.002920
Best KNN	0.005674	0.090818

10.8 Final Classifier Comparison

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Gaussian NB	0.8339	0.7178	0.9532	0.8189	0.9449
Multinomial NB	0.8958	0.9349	0.7906	0.8567	0.9612
Bernoulli NB	0.6482	0.8545	0.1295	0.2249	0.6135
KNN ($k = 5$)	0.9034	0.8848	0.8678	0.8762	0.9444

11 Result Visualization

11.1 Naïve Bayes Comparison

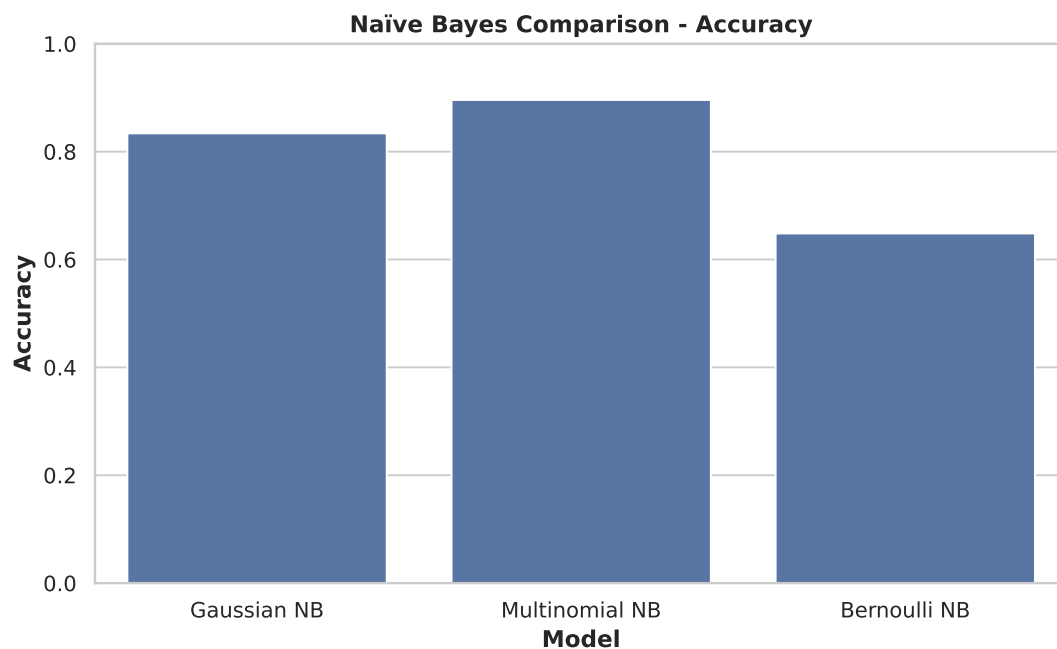


Figure 7: Test-set accuracy comparison across the three Naïve Bayes variants.

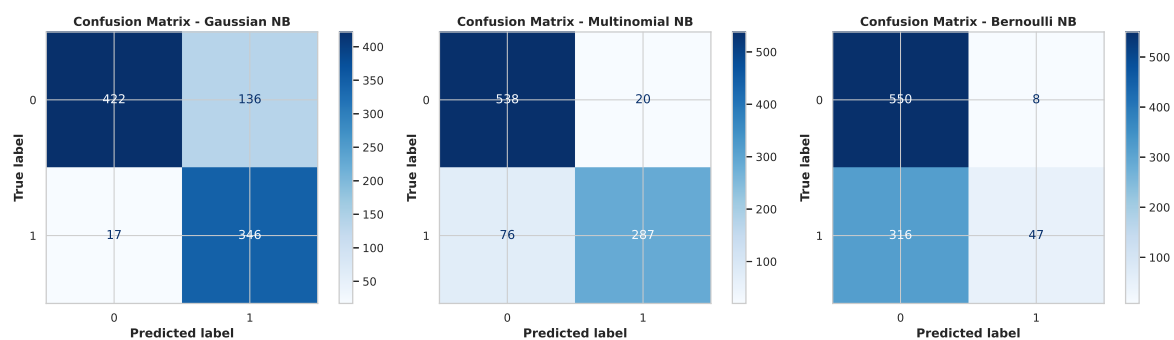


Figure 8: Confusion matrices for Gaussian, Multinomial and Bernoulli NB (left to right).

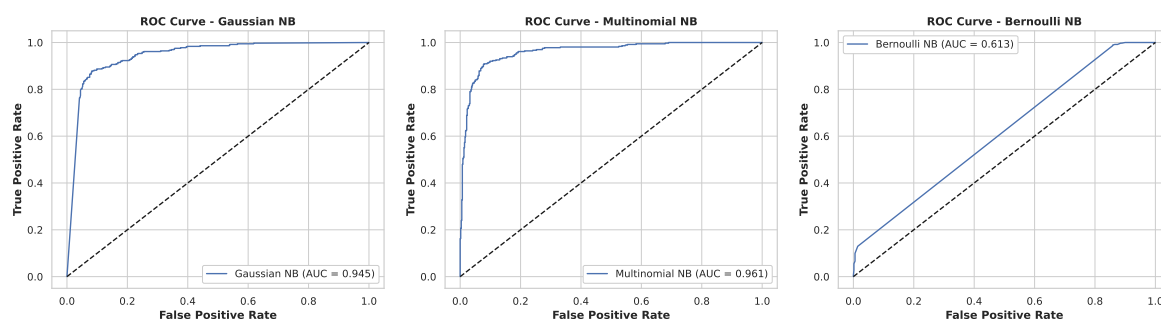


Figure 9: ROC curves for Gaussian, Multinomial and Bernoulli NB (left to right).

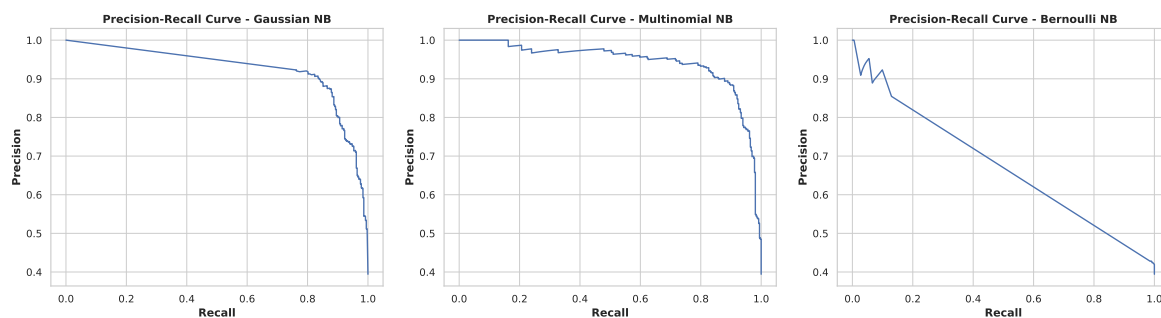


Figure 10: Precision-Recall curves for Gaussian, Multinomial and Bernoulli NB (left to right).

Inference: Multinomial NB comes out on top here – highest accuracy (0.8958), precision (0.9349), and ROC-AUC (0.9612) of the three. That fits the theory from Section 7.1: its multinomial assumption just matches the count/frequency nature of the `word_freq_*` features better than a Gaussian or a hard binary assumption ever could. Gaussian NB, on the other hand, is trading precision for recall – its confusion matrix shows a lot of ham messages getting misclassified as spam, which isn’t great in a real spam filter, since blocking a genuine e-mail is usually worse than letting one spam message through. Bernoulli NB basically falls apart: an ROC-AUC of 0.6135 is barely better than a coin flip, and its 0.6482 accuracy is only a hair above the 60.6% “always predict ham” baseline from Section 4.1. That’s a pretty strong confirmation that the fixed 0.5 binarisation threshold is destroying nearly all the useful signal here (see Section 4.3).

11.2 KNN Performance vs. k

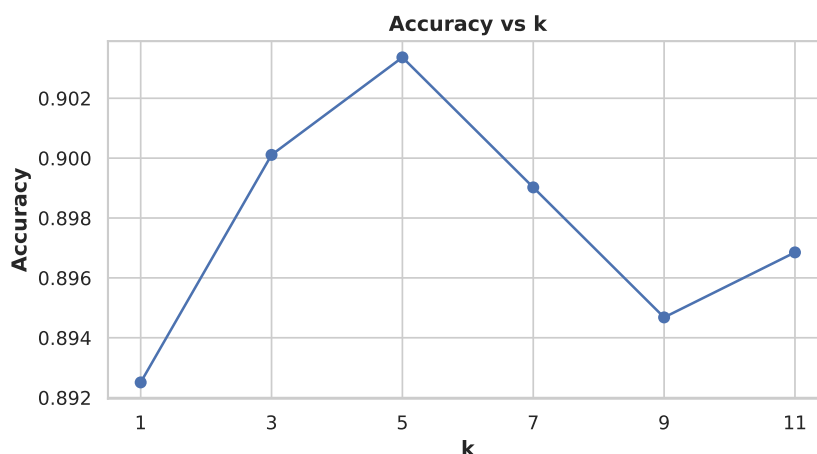


Figure 11: KNN accuracy as a function of k (uniform weighting, default Euclidean distance).

Inference: Accuracy peaks at $k = 5$ (0.9034) and drops off a little for larger k – a fairly textbook bias-variance trade-off. $k = 1$ is the most flexible option and has the highest variance, so it gets dragged down by locally noisy neighbours. Very large k does the opposite – it over-smooths the decision boundary and starts pulling in points from the majority (ham) class, which hurts both accuracy and recall. What’s a bit more interesting is that ROC-AUC keeps climbing with k (0.888 at $k = 1$, up to 0.954 at $k = 11$) even as accuracy falls after $k = 5$. One possible explanation is that larger k gives smoother, better-calibrated probability estimates, even though the

default 0.5 decision threshold stops being the ideal cut-off for accuracy once the neighbourhood gets that large.

11.3 GridSearchCV vs. RandomizedSearchCV

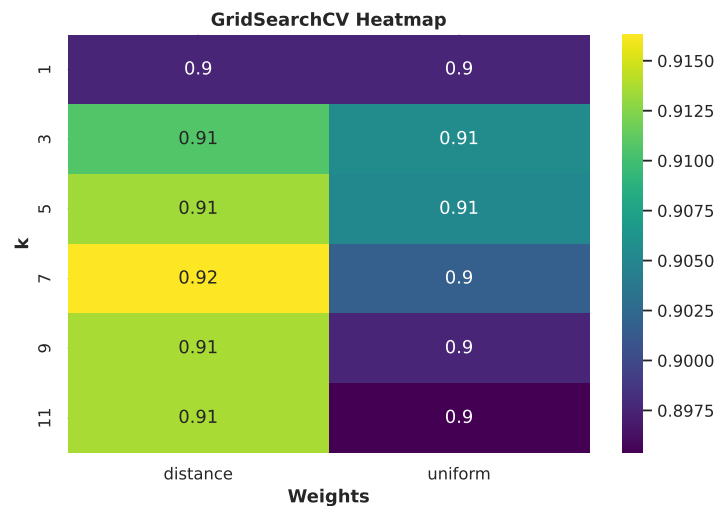


Figure 12: GridSearchCV mean CV accuracy across $n_neighbors$ and $weights$.

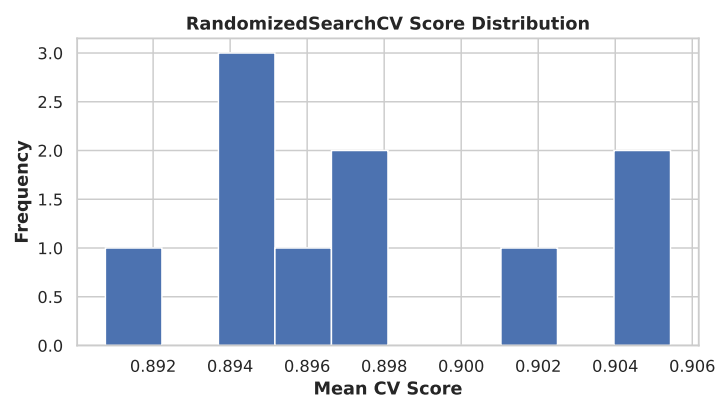


Figure 13: Distribution of mean CV scores across the 10 configurations sampled by RandomizedSearchCV.

Inference: The exhaustive grid search genuinely found a better configuration than either the manual k -sweep (Section 8.2, best test accuracy 0.9034 at $k = 5$, uniform weights) or the random search. Distance weighting paired with Manhattan distance ($p = 1$) at $k = 7$ reached 0.9163 CV accuracy – roughly a percentage point better. Looking at the heatmap, distance weighting beats uniform weighting across almost every value of k we tried, which backs up the idea that closer neighbours really do carry more reliable information for this dataset. RandomizedSearchCV, having only sampled 10 of the full grid’s combinations, ended up with a noticeably weaker setup ($k = 3$, uniform, kd_tree) – about 1.1 points lower in CV accuracy – but it got there in 2.16s versus GridSearchCV’s 18.08s, roughly an $8.4\times$ speed-up. That’s a fairly classic exhaustive-search-vs-speed trade-off.

11.4 KDTree vs. BallTree

Inference: Accuracy is identical for both trees, which isn't surprising – both are exact nearest-neighbour structures, so they have to return the same k neighbours as brute force; they just differ in *how* they search, not *what* they find. BallTree came out faster on both build (0.0137s vs. 0.0223s) and query (0.1795s vs. 0.2151s) for this dataset. This tracks with the theory in Section 7.2: with $d = 57$ dimensions, we're already well past the point where KDTree's axis-aligned splitting starts losing its pruning advantage over brute force – the usual rule of thumb is that KDTrees stop paying off somewhere above roughly 20 dimensions – whereas BallTree's spherical, metric-based partitioning tends to degrade more gracefully.

11.5 5-Fold Cross Validation

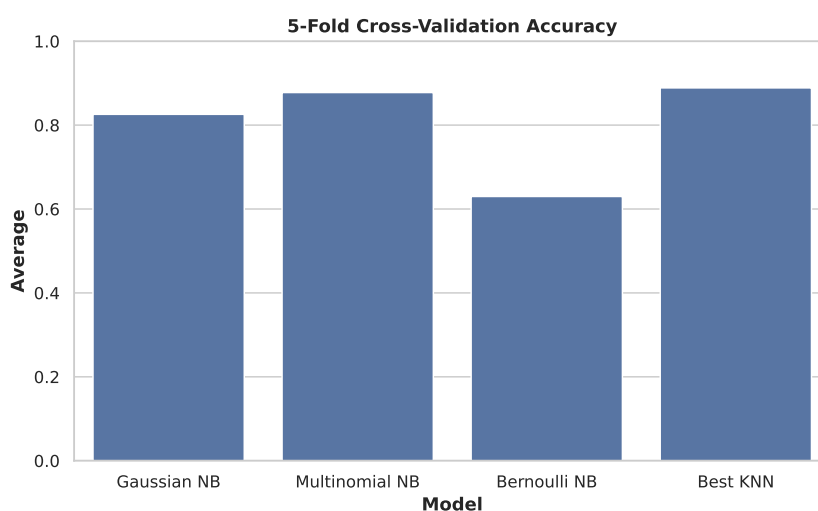


Figure 14: 5-fold cross-validation accuracy comparison.

Inference: The cross-validated ranking (Best KNN > Multinomial > Gaussian > Bernoulli) lines up with the single-split ranking from Sections 8.1–8.2, which is reassuring – it means the earlier result probably wasn't just a lucky split. Fold 5 is noticeably harder for every model; all four scores dip there, which points to that fold containing an intrinsically harder-to-separate subset of e-mails rather than any one model being unstable. That said, Gaussian NB is hit disproportionately hard, falling all the way to 0.70 (a 12–15 point drop compared to its other folds), while Multinomial NB only dips to 0.8533. We think this out-sized sensitivity ties back to Gaussian NB's distributional mismatch discussed in Section 4.3 – whichever fold happens to pick up more of the extreme-tail spam messages is going to violate its normality assumption the hardest.

11.6 Final Tuned KNN – Evaluation

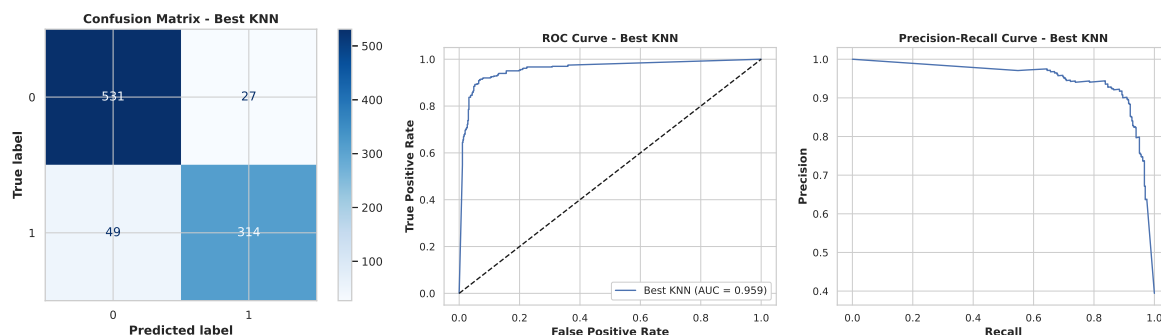


Figure 15: Confusion matrix, ROC curve and Precision-Recall curve for the GridSearchCV-tuned best KNN model ($k = 7$, distance-weighted, Manhattan).

Inference: On the held-out test set, the tuned model reaches 0.9175 accuracy – about 1.4 points above the untuned $k = 5$ /uniform model’s 0.9034 from Section 8.2 – and this is very close to the 0.9163 accuracy GridSearchCV had already estimated via cross-validation (Section 8.3). The gap there is only about 0.1 points, which is a good sign: it suggests the CV procedure generalised well, and the test-set number can be trusted rather than chalked up to a lucky split.

11.7 Experimental Time Analysis

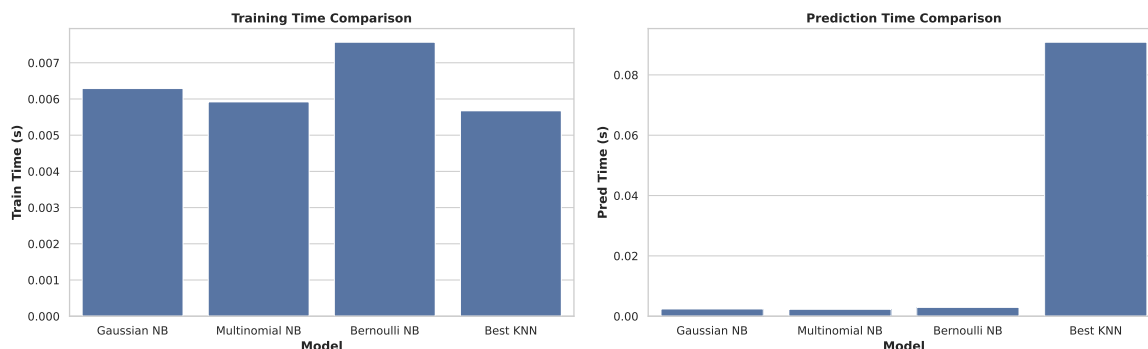


Figure 16: Training time (left) and prediction time (right) across all four final models.

Inference: All four models train in a few milliseconds – Best KNN is actually the fastest of the four to “train” (0.0057s), which makes sense given it’s a lazy learner and really just stores the training data. That matches its theoretical $O(1)$ training complexity from Section 8.8. Prediction tells the opposite story, though: Best KNN’s prediction time (0.0908s) is roughly 32–40 $\times$ slower than any Naïve Bayes variant. That’s because GridSearchCV picked `algorithm='brute'` (Section 8.3), which has to compute the distance to all 3680 training points for every one of the 921 test queries – an $O(nd)$ cost per prediction batch, right in line with what theory predicts. It’s a pretty direct, measured illustration of the training-cost/prediction-cost trade-off between these two algorithm families.

11.8 Final Classifier Comparison

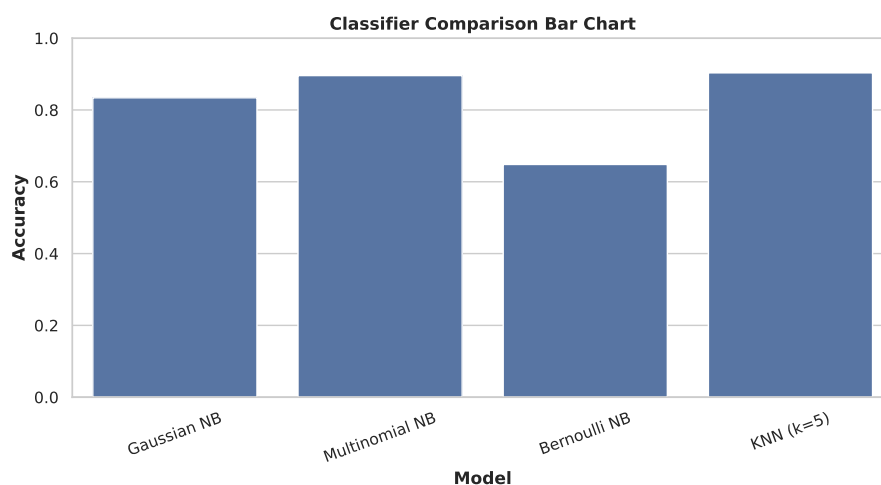


Figure 17: Final test-set accuracy comparison across all four classifiers.

Inference: KNN ($k = 5$) edges out Multinomial NB on raw accuracy and F1, but Multinomial NB still wins on precision and ROC-AUC, and does so at a fraction of KNN’s prediction cost (Section 8.7). So really, which model is “best” depends on what you’re optimising for in deployment – KNN if you want the highest overall correctness, Multinomial NB if you’d rather minimise false alarms and get well-ranked probability scores while spending far less on computation.

12 Additional Tasks

12.1 Different Train-Test Splits

Evaluated using the GridSearchCV-tuned best KNN model ($k = 7$, distance-weighted, Manhattan, brute force):

Split (train/test)	Accuracy	F1-Score
90/10	0.9002	0.8678
80/20	0.9175	0.8920
70/30	0.9247	0.9021

Inference: Both accuracy and F1 improve as more data goes into training – 70/30 gives the tuned KNN model more neighbours to draw on at prediction time – at the cost of a smaller, higher-variance test set to measure that performance against. The trend keeps climbing rather than levelling off, which suggests the model hadn’t fully saturated at 80/20 and could probably benefit from even more training data.

12.2 Euclidean vs. Manhattan Distance

Evaluated at $k = 5$ (uniform weights, `algorithm='auto'`):

Distance	Accuracy	F1-Score
Manhattan ($p = 1$)	0.9045	0.8736
Euclidean ($p = 2$)	0.9034	0.8762

Inference: The two metrics perform almost identically – within 0.1–0.3 points of each other – with Manhattan a touch ahead on accuracy and Euclidean a touch ahead on F1. Given that GridSearchCV also independently landed on $p = 1$ (Manhattan) as part of its best configuration (Section 8.3), this near-tie makes us think the distance-metric choice is a fairly minor factor here compared to k and the weighting scheme.

12.3 Weighted vs. Uniform KNN

Evaluated at $k = 5$:

Weighting	Accuracy	F1-Score
Uniform	0.9034	0.8762
Distance	0.9077	0.8821

Inference: Distance weighting improves both accuracy (+0.43 points) and F1 (+0.59 points) over uniform voting at the same k , which lines up with the same effect that led GridSearchCV to pick distance weighting overall (Section 8.3). It seems like weighting down the influence of farther neighbours mostly helps points sitting near the ham/spam decision boundary, where uniform voting is easier to sway by a majority of merely-nearby-but-not-closest neighbours.

12.4 Overfitting/Underfitting Discussion

The k -sweep in Section 8.2 shows both regimes pretty clearly. $k = 1$ overfits – it follows the training data (noise included) so closely that it’s actually the worst performer on the test set (0.8925 accuracy, lowest ROC-AUC at 0.8882), despite having zero bias toward any particular neighbourhood shape. At the other end, large k (9, 11) starts to underfit with respect to accuracy: the decision rule increasingly just reflects the global class prior (60.6% ham) instead of local structure, so recall for the minority spam class drops (0.8540 at $k = 9$, 0.8512 at $k = 11$, versus 0.8678 at the optimal $k = 5$). $k = 5$ looks like the closest thing to a bias-variance sweet spot for raw accuracy on this feature space, though Section 8.3 shows that once distance weighting enters the picture, that sweet spot shifts slightly, to $k = 7$.

13 Discussion

Analysis Questions

1. **Which Naïve Bayes variant performed best?** Multinomial NB, by a fair margin – highest accuracy (0.8958), precision (0.9349) and ROC-AUC (0.9612) of the three (Section 8.1). Its count-based assumption just matches the `word_freq_*` features much better than either Gaussian NB’s normality assumption or Bernoulli NB’s hard binarisation.
2. **What is the optimal value of k ?** In the plain uniform-weighted sweep, $k = 5$ gave the best test accuracy (0.9034, Section 8.2). Once distance weighting and the Manhattan metric are also tuned, though, GridSearchCV found $k = 7$ to be jointly optimal (CV accuracy 0.9163, Section 8.3) – so really, “best k ” depends on the weighting scheme you’re using.

3. **Compare GridSearchCV and RandomizedSearchCV.** GridSearchCV exhaustively worked through the full 36-combination grid and found a genuinely better configuration (0.9163 CV accuracy) than RandomizedSearchCV's 10 sampled combinations (0.9054) – but it took roughly $8.4\times$ longer to do it (18.08s vs. 2.16s, Section 8.3).
4. **Compare KDTree and BallTree.** Same accuracy for both, since they're both exact search structures, but BallTree was quicker to build and query on this 57-dimensional dataset (Section 8.4), which fits with KDTree's known weakness in higher dimensions.
5. **Compare theoretical and practical complexity.** The measured times lined up well with theory: KNN's near-zero training time confirmed the expected $O(1)$ training complexity, and its noticeably slower prediction time (0.0908s, roughly $32\text{--}40\times$ the Naïve Bayes models) confirmed the $O(nd)$ brute-force prediction cost, since GridSearchCV's chosen model used `algorithm='brute'` (Section 8.7).
6. **Which classifier is preferred for large datasets?** Naïve Bayes, and Multinomial NB in particular, scales better – its prediction cost is $O(d)$ and doesn't depend on the number of training points n , while brute-force KNN's prediction cost grows linearly with n . Even tree-based KNN still carries a much bigger constant factor in practice (Section 8.7), so for very large datasets, Multinomial NB's small hit in accuracy/ROC-AUC compared to KNN is usually a reasonable price to pay for the big gain in prediction throughput.

Stepping back from the specific questions, two things stood out to us over the course of this experiment. First, feature-distribution assumptions seem to matter a lot more than search-strategy tuning – the single biggest swing in this whole report was between Naïve Bayes variants (0.6482 up to 0.8958 accuracy, purely from which likelihood model was assumed), while even the most exhaustive KNN hyperparameter search only bought about 1.4 points over a simple k -sweep. Second, accuracy alone would have completely hidden Bernoulli NB's near-random ranking ability (ROC-AUC 0.6135) – tracking ROC-AUC and the Precision-Recall curve throughout was what actually caught that failure, given the class imbalance we noted back in Section 4.1.

14 Conclusion

We built, tuned, and compared four classifiers – Gaussian NB, Multinomial NB, Bernoulli NB and KNN – on the 4601-sample Spambase dataset. Multinomial NB turned out to be the strongest Naïve Bayes variant, and a distance-weighted, Manhattan-metric KNN with $k = 7$ (found via GridSearchCV) ended up the overall strongest single model on the test set (0.9175 accuracy) – narrowly ahead of Multinomial NB on accuracy, but behind it on precision and ROC-AUC, and roughly $32\text{--}40\times$ slower to query. KDTree and BallTree gave identical accuracy but different speeds, with BallTree the better pick at this dataset's dimensionality. GridSearchCV beat RandomizedSearchCV on accuracy, though at a considerable time cost, and 5-fold cross-validation confirmed these rankings held up across different splits rather than being an artefact of one particular train/test partition. All told, choosing between the Naïve Bayes family and KNN for a real spam-filtering deployment should come down to more than raw accuracy – prediction latency and the relative cost of false positives versus false negatives matter just as much.

15 References

1. Scikit-learn Developers, “Scikit-learn: Machine Learning in Python – Documentation,” [Online]. Available: <https://scikit-learn.org/stable/documentation.html>.
2. A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*, 3rd ed. Sebastopol, CA, USA: O’Reilly Media, 2022.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.
4. M. Hopkins, E. Reeber, G. Forman, and J. Suermondt, “Spambase Data Set,” UCI Machine Learning Repository, 1999. [Online]. Available: <https://archive.ics.uci.edu/ml/datasets/spambase>.